

personal notes, 2026

Commands that actually **save you** in **prod.**

Not the basics. The ones nobody teaches but every DevOps engineer quietly uses — for Linux, Docker, and Kubernetes debugging.

Linux

Docker

Kubernetes

Useful Linux commands — **you don't know yet**

lsof

```
lsof -i :8080
lsof -p <pid>
```

*Who is using port 8080? Which files does a process have open?
Essential when a port "is already in use."*

ss

```
ss -tulnp
ss -s
```

Faster replacement for netstat. Shows open sockets, ports, and which process owns them.

strace

```
strace -p <pid>
strace -e trace=network ls
```

Trace every system call a process makes. Insane for debugging silent failures — "why does this just hang?"

nsenter

```
nsenter -t <pid> -n ip a
```

*Enter the network namespace of a process (or container).
Debug container networking from the host, no shell needed inside.*

watch

```
watch -n 1 'df -h'
watch -d kubectl get pods
```

Re-runs a command every N seconds. Poor man's live dashboard. The -d flag highlights what changed.

dmesg

```
dmesg -T | tail -50
dmesg | grep OOM
```

Kernel ring buffer. First stop when a container or process gets killed unexpectedly — OOM killer logs here.

Most people reach for Google. These people reach for **lsof and strace first.**

Linux commands **inside** Docker workflows

Namespace & cgroup tools

unshare

```
unshare --pid --fork bash
```

Manually create namespaces — exactly what Docker does internally. Great to understand isolation at the kernel level.

cat /proc/<pid>/cgroup

```
cat /proc/$(pgrep nginx)/cgroup
```

See which cgroup a process belongs to. Useful when diagnosing why a container is throttled or OOM-killed.

Docker-specific power commands

docker inspect

```
docker inspect <ctr> | jq '.[0].NetworkSettings'
```

Full JSON metadata of a container — IP, mounts, env vars, restart policy. Pipe to jq to extract what you need.

docker stats

```
docker stats --no-stream  
docker stats <name>
```

Live CPU/memory/net/IO per container. The --no-stream flag gives a one-shot snapshot instead of live feed.

docker diff

```
docker diff <container_id>
```

Shows every file changed inside a running container vs its image. Perfect for debugging "what did this thing write?"

docker cp

```
docker cp <ctr>:/app/logs .
```

Pull files out of a container to your host without exec. Works even when the container has no shell.

docker diff — I used this exactly once and it saved two hours of head-scratching.

K8s commands **everyone skips**

kubectl get events

```
kubectl get events -n <ns> --sort-by='.lastTimestamp'
```

Sorted cluster events. Shows OOMKills, ImagePullBackOff reasons, scheduling failures — before you even look at logs.

kubectl top

```
kubectl top pods -n <ns>  
kubectl top nodes
```

Real-time CPU and memory per pod/node. Requires metrics-server. First command when something feels "slow."

kubectl rollout

```
kubectl rollout status deploy/app  
kubectl rollout undo deploy/app
```

Watch a rollout in real time. One-command rollback to the previous version when a deploy goes sideways.

kubectl diff

```
kubectl diff -f deployment.yaml
```

Shows what will change before you apply. Like git diff but for your cluster state. Use it every time.

kubectl api-resources

```
kubectl api-resources --verbs=list
```

Lists every resource type in the cluster. Useful when working with CRDs or when you forget the exact resource name.

kubectl auth can-i

```
kubectl auth can-i create pods  
kubectl auth can-i '*' '*'
```

Check your RBAC permissions before wasting time debugging access errors. Instant yes/no answer.

kubectl get events --sort-by — should be the first thing you run. Not the last.

Deep K8s **debugging** commands

Pod & container forensics

kubectl debug

```
kubectl debug -it <pod> --image=busybox --target=app
```

Attaches an ephemeral debug container to a running pod — no restart needed. Your best friend when a pod has no shell.

kubectl logs --previous

```
kubectl logs <pod> -p --tail=100
```

Logs from the previous crashed container instance. The only way to see why a CrashLoopBackOff actually happened.

kubectl describe

```
kubectl describe pod <pod> | grep -A5 Events
```

The Events section at the bottom tells you exactly why a pod is Pending, Evicted, or stuck. Always scroll to the bottom.

kubectl exec

```
kubectl exec -it <pod> -c <container> -- /bin/sh
```

Exec into a specific container in a multi-container pod. Forget the -c flag and you'll always hit the wrong one.

Network debugging inside the cluster

kubectl run netshoot

```
kubectl run tmp --rm -it --image=nicolaka/netshoot -- bash
```

Spin up a throwaway pod with every network tool installed — curl, dig, nslookup, tcpdump, iperf. Debug DNS from inside the cluster.

kubectl port-forward

```
kubectl port-forward svc/myapp 8080:80
```

Forward a service or pod port to localhost. Fastest way to test an app without exposing it or touching Ingress.

nicolaka/netshoot — bookmark this image. It's a Swiss Army knife in a 150MB container.

Resource, config & RBAC debugging

Config & secrets

kubectl get secret -o jsonpath

```
kubectl get secret mysecret -o  
jsonpath='{.data.password}' | base64 -d
```

Decode a secret value directly. No more manual base64 copy-paste from the describe output.

kubectl set env

```
kubectl set env deploy/app DEBUG=true
```

Inject or update an env var on a deployment without editing YAML. Triggers a rolling restart automatically.

Resource & scheduling

kubectl get pods -o wide

```
kubectl get pods -o wide -n <ns>
```

Shows which node each pod landed on. First thing to check if a pod is Pending — is the node overloaded or tainted?

kubectl cordon / drain

```
kubectl cordon <node>  
kubectl drain <node> --ignore-daemonsets
```

Mark a node unschedulable (cordon) and evict its pods (drain) safely before maintenance. Essential for node-level ops.

kubectl label / taint

```
kubectl label node <node> env=prod  
kubectl taint node <node> key=val:NoSchedule
```

Control which pods land on which nodes. Taints repel pods; labels attract them via nodeSelector or affinity rules.

kubectl get hpa

```
kubectl get hpa -n <ns> -w
```

Watch HorizontalPodAutoscaler in real time. See current replicas vs desired and what metric is driving scaling decisions.

A pod stuck on Pending for no reason? **Check node taints first. Always.**

Commands that **work better together**

journalctl + grep

```
journalctl -u kubelet -f | grep -i error
```

Stream kubelet logs live with error filtering. Way faster than reading the whole log dump.

kubectl get all + jq

```
kubectl get pods -o json | jq '.items[] | select(.status.phase=="Pending") | .metadata.name'
```

Filter only Pending pods by name using jq. Scriptable, composable, beats grep on raw output.

docker logs + grep

```
docker logs -f <ctr> 2>&1 | grep -i "error\|warn"
```

Stream container logs with error/warn filtering. The 2>&1 catches stderr too — critical for app logs.

watch + kubectl top

```
watch -n 2 kubectl top pods -n prod
```

Live pod resource usage refreshed every 2s. Poor man's Grafana when you don't have a dashboard open.

lsof + kill

```
lsof -i :8080 | awk 'NR>1{print $2}' | xargs kill -9
```

Find and kill whatever is holding port 8080. One liner. No manual PID hunting.

kubectl logs --all-containers

```
kubectl logs <pod> --all-containers=true -f
```

Stream logs from every container in a multi-container pod simultaneously. One command, full picture.

These combos are not in the docs. **They live in ~/.bash_history of people who've been paged at 3am.**



You don't need **more tutorials.** You need reps.

Open your terminal. Run these. Break something on purpose.

That's how these commands actually stick.

Which one did you not know? Drop it below

github.com/Heyyprakhar1